

Information Retrieval and Data Mining (COMP0084)

Coursework (2025/26)

Dr Vasileios Lamos (v.lamos@ucl.ac.uk)

Marking penalties specific to this coursework

Marking penalties are denoted with **bold font** throughout the specification of the coursework. The maximum amount of marks that can be lost due to penalties (= **20**) is the sum of the following:

- **4** marks for malformed submission file names and / or imposing a directory structure (inside your source code or in the submitted files),
- **5** marks for a report that did not follow the formatting guidelines,
- **1** mark for having figures that are not in vector graphics,
- **5** marks for incorrectly formed output files (**1** mark per output file), and
- **5** marks for very slow to execute source code files (**1** mark per source code file).

Penalty marks are subtracted from your final mark. To avoid the majority of the aforementioned penalties, please follow the coursework's guidelines as described below.

Task definition

An information retrieval model is an essential component for many applications such as search, question answering, and recommendation. In this coursework you are going to develop information retrieval models that solve the problem of passage retrieval, i.e. given a text query, we need to return a ranked list of short texts (passages). More specifically, in the first parts of this assignment (Tasks 1 to 4) you are going to build a passage re-ranking system: given a candidate list of returned passages for a text query, you should re-rank these returned passages based on an information retrieval model. In the second part (Task 5), you are going to evaluate a retrieval model and develop your own (basic) model using machine learning.

qid	pid	query	passage
523270	2818345	toyota of plano plano tx	DART's Red Line runs along North Central...
527433	1537731	types of dysarthria from cerebral palsy	In some cases, further testing will also be...
1113437	5194230	what is physical description of spruce	Source: *U.S. Rehab Aide Job Description...
833860	5043973	what is the most popular food in switzerland	The national currency in Switzerland is the...
1056204	2328890	who was the first steam boat operator	a relatively small usually open craft of a...

Figure 1: Sample rows from the `candidate-passages-top1000.tsv` file.

Data

Data can be downloaded from [this online repository](#). The data set consists of 5 files:

- `test-queries.tsv` is a tab separated file, where each row contains a test query identifier (`qid`) and the actual query text.
- `passage-collection.txt` is a collection of passages, one per row.
- `candidate-passages-top1000.tsv` is a tab separated file with an initial selection of at most 1000 passages for each of the queries in `test-queries.tsv`. The format of this file is `<qid pid query passage>`, where `pid` is the identifier of the passage retrieved, `query` is the query text, and `passage` is the passage text (all tab separated). The passages contained in this file are the same as the ones in `passage-collection.txt`. However, there might be some repetitions, i.e. the same passage could have been returned for more than 1 query. Figure 1 shows some sample rows from that file.
- `train-data.tsv` and `validation-data.tsv` are similarly formatted data sets that you will use **only in Task 5** for training and validating models. They have an additional relevance column indicating the relevance of a passage to a query. The training data should be used to train models and the validation data to evaluate/test their performance.

It is important that you do **not** change the above filenames, i.e. your source code must use the above filenames, otherwise we might not be able to assess your code automatically and a **penalty** of 5 marks will be applied.

Important guidelines

Please read carefully as all deliverables (**D1** to **D15**) are described within the tasks. If a deliverable requests an answer in a specific format, follow these guidelines carefully as parts of the marking will be automated.

Although we strongly suggest using Python, you could also use Java. Please make one consistent choice for this coursework. Do not submit your source code as a Jupyter Notebook or similar. **All source code must be your own.** You are not allowed to reuse any code that is available from someone or somewhere else. Do not use external functions or libraries that can solve (end-to-end) the tasks of building an inverted index, TF-IDF, the vector space model, BM25, or any of the query likelihood language models. You could use functions or libraries that enhance numeric operations, data storage, and retrieval when this does not violate the aforementioned conditions.

Please make sure that your code is not very inefficient. Very inefficient submissions will be penalised. The expected completion time also depends on computing power. The maximum runtime per task should not exceed 60 minutes. Otherwise, a **penalty** of 1 mark for each task that is slow will be applied.

With the exception of **D14**, please use only unigram (1-gram) text representations to solve this coursework's tasks.

Use the [ACL L^AT_EX template to write your report](#) and submit 1 PDF file named `report.pdf`. Your report should not exceed 7 pages. If the report's format violates these rules, a **penalty** of 5 marks will be applied.

All plots/figures in your report must be in vector graphics format (e.g. `.pdf`).¹ If they are not, a **penalty** of 1 mark will be applied.

¹Vector graphics, en.wikipedia.org/wiki/Vector_graphics

In total, your submission should have 5 source code files (`task[1-5].py/java`), an optional `requirements.txt` file, 5 output CSV files (`tfidf.csv`, `bm25.csv`, `laplace.csv`, `lidstone.csv`, `dirichlet.csv`), and 1 PDF file with the report (`report.pdf`). Do not deploy any internal directory structure in your submission, i.e. just submit the aforementioned 11 or 12 files. Do not compress (e.g. `zip`) your submission files. The data files should not be included in your submission. You should assume that the data files will be placed (by us) in the same directory as the source code (no directory structure) when we will assess/run your source code. If you impose a directory structure, then we will need to remove it by editing your source code to then be able to assess your submission automatically. This will result in a 5 mark **penalty**. Your source code may generate intermediate files when it runs (e.g. to transfer an outcome of a previous step to the next one). These are OK or essential, but should be removed from your submission.

Tasks and deliverables

Task 1 – Text statistics (20 marks). Use the data in `passage-collection.txt` for this task. Extract terms (1-grams) from the raw text. In doing so, you can also perform basic text preprocessing steps. However, you can also choose not to.

- D1** Do not remove stop words in the first instance. Describe and justify any other text preprocessing choice(s) if any [2 marks], and report the size of the identified index of terms (vocabulary) [1 mark].

Then, implement a function (or a block of source code) that counts the number of occurrences of terms in the provided data set, plot (Figure 1) their probability of occurrence (normalised frequency) against their frequency ranking [1 mark], and qualitatively justify that these terms follow Zipf’s law [1 mark].

As a reminder, Zipf’s law for text sets $s = 1$ in the Zipfian distribution defined by

$$f(k; s, N) = \frac{k^{-s}}{\sum_{i=1}^N i^{-s}}, \quad (1)$$

where $f(\cdot)$ denotes the normalised frequency of a term, k denotes the term’s frequency rank in our corpus (with $k = 1$ being the highest rank), N is the number of terms in our vocabulary, and s is a distribution parameter. How does your empirical distribution compare to the actual Zipf’s law distribution? Use Eq. 1 to explain and to quantify where their difference is coming from [12 marks], and also compare the two in a log-log plot (Figure 2) [1 mark].

How will the difference between the two distributions be affected if we also remove stop words? Justify your answer by depicting (Figure 3) and quantifying this difference. [2 marks]

- D2** Submit your source code for Task 1 as a single Python or Java file titled `task1.py` or `task1.java`, respectively.

Task 2 – Inverted index (10 marks). Use `candidate-passages-top1000.tsv` for this task (unique instances of column pairs `pid` and `passage`). Using the vocabulary of terms identified in Task 1 (you will need to choose between removing or keeping stop words), build an inverted index for the collection so that you can retrieve passages in an efficient way [3 marks].

D3 Provide a description of your approach to generating an inverted index, report the information that you decided to store in it, and justify this choice. [7 marks]

D4 Submit the source code for Task 2 as a single Python or Java file titled `task2.py` or `task2.java`, respectively.

Task 3 – Retrieval models (25 marks). For this task, please use `test-queries.tsv` and `candidate-passages-top1000.tsv`. There is no need to describe your approach in the report. You will be marked based on your output only. If the output does not follow the description (see below), we will apply a **penalty** of 1 mark per problematic output file and attempt to fix minor issues manually (e.g. remove headers, correct spacing etc.). However, if after fixing minor issues, the output file does not have the exact number of rows as requested (this means that there is a fundamental flaw in your solution), you will receive 0 marks for that part of the coursework. We will not be able to re-arrange the order of rows that you have provided.

D5 Extract the TF-IDF vector representations of the passages using the inverted index you have constructed. Using the IDF representation from the corpus of the passages, extract the TF-IDF representation of the queries. Use a basic vector space model with TF-IDF and cosine similarity to retrieve at most 100 passages from within the 1000 candidate passages for each query (some queries have fewer candidate passages). Store the outcomes in a file named `tfidf.csv` [15 marks]. Each row of `tfidf.csv` must have the following format:

`qid,pid,score`

where `qid` denotes the query's identifier, `pid` denotes the passage identifier, and `score` is their cosine similarity score. Note that there should be no spaces between commas. The output `.csv` files should not have headers. Strictly report the identifiers from the raw data (do not replace them with your own). `qid,pid` pairs should be ranked by similarity score in descending order, i.e. starting with the highest similarity score. You should first report the top `qid,pid` pairs for one `qid`, then move on to the next, following the same query order as in the `test-queries.tsv` file. The same file format should be used for reporting the outputs of all the retrieval or language models that you will implement as part of this coursework (i.e. this applies to Task 4 as well). The last column should report the corresponding similarity/relevance score of each model. Each output `.csv` file is expected to have exactly 19,290 rows.

D6 Use the inverted index to also implement BM25 while setting $k_1 = 1.2$, $k_2 = 100$, and $b = 0.75$. Retrieve at most 100 passages from within the 1000 candidate passages for each test query. Store the outcomes in a file named `bm25.csv`. [10 marks]

D7 Submit the source code for Task 3 as a single Python or Java file titled `task3.py` or `task3.java`, respectively.

Task 4 – Query likelihood language models (20 marks). Use `test-queries.tsv` and `candidate-passages-top1000.tsv` for this task. Implement the query likelihood language model with (a) Laplace smoothing, (b) Lidstone correction with $\epsilon = 0.1$, and (c) Dirichlet smoothing with $\mu = 50$, and retrieve at most 100 passages from within the 1000 candidate passages for each test query. There is no need to describe your approach in implementing these language models in the report; you will be marked based on your output only. However, please note that D11 needs to be answered in the report. Similarly to Task 3, if the output does not

follow the description (see Task 3), we will apply a **penalty** of 1 mark per problematic output file and attempt to fix minor issues manually (e.g. remove headers, correct spacing etc.). If after fixing minor issues, the output file does not have the exact number of rows as requested (this means that there is a fundamental flow in your solution), you will receive 0 marks for that part of the coursework. We will not be able to re-arrange the order of rows that you have provided.

D8-10 Store the outcomes of (a), (b), and (c) in the files `laplace.csv`, `lidstone.csv`, and `dirichlet.csv`, respectively. In these files, the 3rd column should report the natural logarithm of the probability score. [12 marks]

D11 Which language model do you expect to work better and why? Give a few empirical examples based on your data and experiments. The examples need to be comprehensive to a reader of the report that does not have access to the data (i.e. do not refer to passage or query IDs). [2 marks]

Which language models are expected to be more similar and why? Give a few empirical examples based on your data and experiments. [2 marks]

Comment on the value of $\epsilon = 0.1$ in the Lidstone correction. Is this a good choice, would there be a better setting (if so, please provide a range of values), and why? [2 marks]

If we set $\mu = 5000$ in Dirichlet smoothing, would this be a more appropriate value, and why? [2 marks]

D12 Submit the source code for Task 4 as a single Python or Java file titled `task4.py` or `task4.java`, respectively.

Task 5 – Retrieval quality evaluation and machine learning (25 marks).

D13 Implement your own methods to compute the average precision and NDCG metrics. Optimise BM25's parameters using the training data (`training-data.tsv`). Then, evaluate the performance of BM25 on the validation data (`validation-data.tsv`). Report your approach and the obtained performance. [10 marks]

D14 Represent queries and passages using legacy word embeddings (word2vec, fastText or ELMo).² Query/passage embeddings should be computed by averaging the embeddings of their words/tokens. Use these word embedding representations to implement a logistic regression model (classifier) that predicts the relevance of a passage to a given query (binary classification task). Train your model using `training-data.tsv` and test your model on `validation-data.tsv`. Describe the entire process and report the performance using the metrics in D13. Analyse the effect of different learning rates on the model's performance. Please note that the logistic regression classifier should be your own implementation (do not use an external implementation for this). [15 marks]

D15 Submit the source code for Task 5 as a single Python or Java file titled `task5.py` or `task5.java`, respectively. Please note that your source code should print out the model performance(s) described in D13 and D14.

²Word embedding representations can be obtained from external resources [such as this one](#).